TP Développement Embarqué

1. Présentation

Le but de ce TP est d'effectuer une prise en main du développement embarqué sur Raspberry Pi, notamment les aspects traitement d'image, traitement parallèle et contraintes de temps réel.

Cette découverte se fera à l'aide de codes d'exemples qui vous seront fournis et qui seront directement exécutés sur la carte Raspberry Pi.

Note importante : Les codes fournis ne sont donnés qu'à titre d'exemple, il ne s'agit pas de solutions idéales fournies pour l'Enseignement d'Intégration (EI), même s'ils peuvent servir de point de départ. D'autres pistes peuvent également être trouvées en parcourant les documentations qui seront référencées dans ce document.

2. Traitement d'image sur Raspberry Pi

Lors de l'EI, l'un des objectifs sera d'assurer la navigation autonome du robot sur un circuit constitué de pistes blanches sur un fond sombre. Le traitement des images provenant de la caméra est alors primordial pour estimer la position du robot par rapport aux pistes pour contrôler au mieux les moteurs.

Cette partie présente certains éléments essentiels pour agir sur des images en utilisant Python sur Raspberry Pi.

A. Environnement logiciel et représentation d'une image

Pour le traitement d'image, la bibliothèque Python de référence est Opencv. Elle apporte un grand nombre de fonctions précompilées qui permettent d'effectuer différentes opérations de façon efficiente. Elle repose sur la bibliothèque spécialisée dans le calcul matriciel, NumPy.

L'environnement d'exploitation du Raspberry Pi est fourni avec Python 3.13.5, Numpy 2.2.4 et Opencv 4.10.0[1].

Pour la suite, nous travaillerons dans le dossier « TP_Dev_Emb » qui se trouve sur le bureau de l'environnement d'exploitation de la Raspberry Pi.

Question 1 : Ouvrez le code Python « code1.py », et lisez-le avant de l'exécuter. Lorsque l'image s'affiche, placez votre curseur dessus et regardez les informations qui s'affichent. Que peut-on en conclure sur la représentation d'une image (organisation des données, nombre de bits par pixels…) ?

B. Traitement d'une image plus réaliste

Nous allons maintenant travailler sur l'image « photo_test.png » qui est un peu plus représentative des images que le robot devra traiter lors de l'EI. L'une des méthodes possibles pour estimer la position du robot par rapport à la piste blanche est de calculer la position du barycentre de cette dernière. Pour cela, il n'est pas recommandé d'effectuer le calcul directement sur l'image brute, qui peut être bruitée ou présenter

[1] https://docs.opencv.org/4.10.0/d6/d00/tutorial_py_root.html

différents éléments qui pourraient être pris en compte dans le traitement. Cela explique l'importance de deux étapes classiques du traitement d'image : le prétraitement (filtrage, restauration et accentuation de l'image…) et la segmentation (seuillage, détection de contours…).

Question 2 : Ouvrez le fichier « code2.py » et analysez-le avant de l'exécuter. Le résultat ici n'est pas satisfaisant, améliorez les choses en modifiant uniquement les paramètres des fonctions de transformations morphologiques (lignes 22 à 24).

Question 3 : Pour faciliter les choses, il peut être pertinent d'améliorer la partie prétraitement de l'image. Modifiez maintenant les paramètres des fonctions de flou et de seuil (lignes 16 et 17). Quels sont leurs impacts sur le résultat final ?

C. Temps de calcul nécessaire

Dans les applications de véhicules autonomes, il est primordial de contrôler le temps de calcul des prises de décision afin d'adapter les commandes des actionneurs en temps réel. Nous allons ici ajouter quelques éléments dans le code pour pouvoir avoir une idée du temps de calcul nécessaire.

Question 4 : Dans « code2.py », ajoutez en ligne 3 : « import time » et en ligne 10 : « startTime = time. perf_counter () ». Il suffira ensuite d'ajouter la ligne : « print(time.perf_counter() – startTime) » à l'endroit souhaité pour afficher le temps écoulé depuis le passage à la ligne 10.
Vous pouvez maintenant observer l'impact des différents paramètres du code sur le temps d'exécution ainsi que celui de différentes fonctions (print, sauvegarde de l'image de sortie…)

D. Mise en application : détection d'intersection

Lors de l'EI, le robot devra détecter les intersections afin de pouvoir les franchir et poursuivre son itinéraire. L'une des méthodes possibles (pas forcément la plus optimale) est de compter le nombre d'angles dans l'image.

Question 5 : Ouvrez « corner_detection.py », analysez-le et modifiez les différents paramètres pour avoir un résultat acceptable. Faites valider par l'encadrant.

E. Utilisation de la caméra

Nous allons maintenant voir comment récupérer les images de la caméra. Le robot est équipé d'une PiCam V2, qui peut être utilisée par un code Python via la bibliothèque Picamera2[2]. Celle-ci permet notamment de configurer plusieurs paramètres de la caméra : résolution, temps d'exposition, format des pixels… L'objectif

[2] https://pip-assets.raspberrypi.com/categories/652-raspberry-pi-camera-module-2/documents/RP-008156-DS-3-picamera2-manual.pdf

des prochaines questions est de comprendre l'impact de différentes méthodes possibles pour convertir une image issue de la caméra en un objet compréhensible par OpenCV.

Question 6 : Ouvrez « perception1.py », analysez-le avant de l'exécuter. Que pouvez-vous conclure de ce temps nécessaire pour la prise d'une photo et de l'impact sur un système temps réel ? Quelle est la cause de cette latence ?

Question 7 : Ouvrez maintenant « perception2.py », observez les différences avec le code précédent et exécutez-le. Faites maintenant quelques tests sur les deux codes en modifiant la résolution de la caméra. Que peut-on en conclure ?

3. Traitement parallèle

Dans cette partie, nous nous intéresserons notamment à deux solutions pour paralléliser des tâches en Python : le multithreading et le multiprocessing.
Un processus est une instance de programme qui s'exécute sur le processeur. Chaque processus est indépendant et dispose donc de son propre espace mémoire réservé.
Les threads, quant à eux, vont partager un même espace mémoire.
Leur fonctionnement ainsi qu'une évaluation de leur performance se fera à l'aide d'une routine de calcul intensif.

A. Routine et influence de la définition de fonction

Question 8 : Ouvrez « routine.py », et lisez-le avant de l'exécuter. Décommentez les lignes 21 à 25 avant de lancer une nouvelle exécution. Le temps nécessaire est-il proportionnel au nombre de tâches à accomplir ?

Question 9 : Commentez maintenant les lignes 15 à 25 avant de décommenter les lignes 3 à 9 ainsi que 27 et 28. Quelle est l'évolution du temps d'exécution ? Comment peut-on expliquer cela ?

B. Multithreading

La notion de multithreading est fréquemment utilisée dans le domaine du calcul parallèle. Cette notion a un impact direct sur les fiches de description commerciale des processeurs où avec la distinction entre cœurs physiques (nombre d'unités de calcul physiquement disponibles) et les cœurs virtuels (nombre de threads pouvant être exécutés en parallèle, en général deux fois le nombre de cœurs physiques).

Pour tester la notion de multithreading en Python, nous allons utiliser la bibliothèque *threading* et créer une classe qui hérite de la classe *Thread* pour pouvoir appeler notre routine. Cela permet d'utiliser la méthode *start()* pour lancer l'exécution des threads en parallèle et la méthode *join()* qui permet de les synchroniser (le fil d'exécution se met en pause le temps que le thread en question se termine).

Question 10 : Ouvrez « multithreads.py » et lisez-le avant de l'exécuter. Comparez le temps d'exécution obtenu, avec celui du code séquentiel utilisant des fonctions. N'hésitez pas à lancer plusieurs fois l'expérience, et à augmenter le nombre de routines à effectuer. Est-ce que ce résultat était attendu ? Comment peut-on expliquer cela ?

Attention : Cette conclusion ne concerne que le multithreading sur Python à cause du fonctionnement de l'interpréteur. La notion de multithreading reste tout à fait valable pour un code compilé ! Son utilisation sur Python peut toutefois être pertinente dans le cas de threads qui gère des I/O (quand un thread attend des entrées extérieures, un autre peut utiliser le temps d'attente pour accomplir une autre tâche.

C. Multiprocessing

Nous allons utiliser une structure de code similaire, mais cette fois-ci avec la librairie multiprocessing.

Question 11 : Ouvrez « multiprocess.py » et lisez-le avant de l'exécuter. Que pensez-vous du temps d'exécution obtenu avec cette solution ? Augmentez progressivement le nombre de processus lancés en parallèle. Comment évolue le temps d'exécution ? Pourquoi ?

Question 12 : Ajouter une nouvelle routine qui demande une autre charge de calcul (par exemple deux fois plus que la routine initiale). Comment se comporte le système quand on chercher à exécuter en parallèle des tâches qui ont des besoins différents en terme de temps d'exécution ?

Question 13 : Proposez un code qui effectue deux traitements d'une image en parallèle : la détection d'intersection et le calcul du barycentre de la piste visible.